Verifier-Assisted versus One-Shot LLM Intent→Configuration: A Paired NetOpsTraceBench Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed NetOpsTraceBench, a paired comparison of a one-shot LLM intent→configuration pipeline and a verifier-assisted generate–validate–repair pipeline on 1,200 intent→config episodes. Using gpt-5-mini, a deterministic NetOps validator, and Mininet-style emulation, we applied a preregistered binary episode success rule (validator accepts final config AND emulator shows no availability degradation above tolerance AND no automatic rollback). Of 1,200 planned paired tasks we completed 1,199 pairs. Baseline success = 1,196/1,199 (0.9974979149); guarded success = 1,199/1,199 (1.0000000000). The paired-success-rate difference (guarded - baseline) = 0.0025020851 (95% bootstrap percentile CI [0.0000, 0.0058381985], bootstrap_seed=1729, resamples=10,000); exact McNemar two-sided p = 0.25. The preregistered power target sought a 5 percentage-point minimum detectable effect; given the observed near-ceiling baseline, the study lacked power to detect much smaller absolute gains. Representative validator traces and provider audit logs show repair was invoked only three times. Under these controlled emulation and task-corpus conditions we find no statistically significant advantage for the verifier-assisted pipeline; we report artifacts, analytic deviation (Wilcoxon → exact McNemar for binary outcomes), and detailed execution accounting to support reproducible interpretation.**

I. Introduction

Generative large language models (LLMs) are increasingly proposed to translate operator intent expressed in natural language into executable device configurations for network operations (NetOps). Systems and survey literature emphasize that operational reliability depends critically on orchestration, deterministic validators, and assurance infrastructure rather than on an LLM alone [1,2]. Generate–validate–repair (verifier-assisted) architectures are a canonical mitigation strategy: an LLM proposes candidate outputs, an automated verifier checks constraints and safety, and an automated repair loop iterates until the candidate satisfies verifier rules [3–5]. Despite conceptual support, publicly archived, preregistered, paired evidence measuring whether verifier assistance yields an operationally meaningful improvement (rollback-aware binary success) relative to one-shot generation is sparse.

We address this gap with NetOpsTraceBench: a preregistered, paired, emulator-grounded study comparing (A) baseline one-shot NL→config generation and (B) a guarded generate–validate–repair pipeline using the same shared initial candidate. The study asked: Can a reproducible, workflow-centered benchmark of paired intent→config episodes produce an objective paired binary success metric that reliably distinguishes verifier-assisted pipelines from one-shot generation across two simulated topologies? Our contributions are (1) a preregistered experimental design and full execution artifacts (study ID rX4f7-1); (2) a paired analysis on 1,199 completed pairs using gpt-5-mini and a deterministic validator; (3) transparent reporting of an analytic deviation (final confirmatory test used an exact McNemar test appropriate for paired binary outcomes) and execution accounting that explains why the guarded pipeline rarely required repair.

II. Related Work

Recent surveys and system papers position LLMs within AIOps/NetOps pipelines and emphasize the importance of verifiers, provenance, and end-to-end workflow evaluation [1,6]. Work on verifier-guided NL→formal translations and intent→topology compilation has shown the feasibility of generate–validate-repair approaches in domains such as security topology compilation and CTL/specification generation [3,7]. Empirical and conceptual studies document LLM failure modes relevant to NetOps—hallucination, incorrect semantics/units, overconfidence, provenance loss, and prompt injection—which motivate guarded architectures and runtime auditing [8–10]. Several recent prototypes demonstrate LLM-assisted configuration generation and misconfiguration detection but typically evaluate on constructed or small held-out sets [4,5,11]. That literature motivates workflow-centered evaluation metrics (trace quality, rollback measurement, canaries) rather than one-shot QA metrics; NetOpsTraceBench operationalizes a paired, rollback-aware binary success metric in a preregistered framework and archives the run artifacts for reproducibility.

III. Methodology

Preregistration and scope: The study preregistration (rX4f7-1; preregistration SHA-256 = 77abe8b493bde44484e0b3d0 2cfd7f9ec1621ff7e7847889082ae177a7ed5712) froze task selection, inclusion/exclusion rules, contamination thresholds, stopping rules, seeds, analysis plan, and failure treatments before execution. NetOpsTraceBench_v1 comprises 1,200 paired intent→config tasks split into two topology clusters (edge = 600, core = 600). Each task is public synthetic or consented with deterministic provenance recorded. Tasks are vendor-agnostic canonical sequences rendered into device configurations for emulator execution. Inclusion/exclusion rules were deterministic: pairs were excluded if contamination checks flagged training-data overlap above thresholds or if both episodes failed to produce parsable vendor renderings after one automated re-execution attempt; exclusions are logged and reported.

Pipelines and instrumentation: We compared (A) Baseline one-shot — a single LLM call producing an initial candidate configuration; (B) Guarded verifier-assisted — the same initial candidate plus at most one automated repair iteration triggered when the deterministic validator rejected the candidate. Execution used requested_model = gpt-5-mini (provider recorded as openai_responses) orchestrated by adapter hosted_netops_gvr_v1. Deterministic_netops_validator_v5 handled parsing, intent-constraint checks, and emulator preflight; the task generator was deterministic_netops_task_generator_v5. The execution manifest records model_calls_used = 1,203 and maximum_model_calls = 2,400.

Success metric and confirmatory analysis: The preregistered binary episode success rule required (1) validator acceptance of the final configuration, (2) emulator execution with no availability degradation above the preregistered tolerance, and (3) no automatic rollback during simulated execution. The preregistration specified a nonparametric paired procedure (Wilcoxon signed-rank) for paired outcomes; because realized outcomes are binary, the archived confirmatory analysis used an exact McNemar two-sided test on the paired 2×2 contingency table and computed a paired bootstrap percentile 95% CI for the paired success-rate difference using frozen bootstrap_seed = 1729 and 10,000 resamples. This post-preregistration analytic choice is documented in archived artifacts (analysis/paired_contingency_table.csv, analysis/results.json, analysis/analysis_log.jsonl). The primary estimand is paired_success_rate_difference_guarded_minus_baseline. The preregistered power plan targeted a minimum detectable effect (MDE) of 5 percentage points (absolute) at two-sided α=0.05 and power 0.8 given assumed baseline $\approx$0.70; the power rationale and sensitivity scenarios are recorded in the preregistration.

IV. Results

Execution and pair accounting: The run completed 2,398 of 2,400 planned episodes and produced 1,199 complete paired observations (completed_pairs = 1,199; failed_episode_count = 2). The analysis used complete_pair_count = 1,199. Provider audit and orchestration logs record hosted_model_call_event_count = 1,203 (completed_hosted_model_call_event_count = 1,202; failed_hosted_model_call_event_count = 1) and stage_counts indicating shared_initial_generation = 1,200 and repair = 3. These counts show that a single shared initial generation was produced per task and that the guarded pipeline invoked repair calls only three times across the entire corpus.

Primary confirmatory outcomes (paired contingency and rates): The paired 2×2 contingency table (analysis/paired_contingency_table.csv) is: n11 = 1,196 (both methods succeeded), n10 = 3 (guarded succeeded while baseline failed), n01 = 0 (baseline succeeded while guarded failed), n00 = 0 (both failed). From these counts: baseline_success_count = 1,196 (baseline_success_rate = 1,196/1,199 = 0.9974979149291076) and guarded_success_count = 1,199 (guarded_success_rate = 1,199/1,199 = 1.0). The observed paired-success-rate difference (guarded - baseline) = 0.002502085070892446.

Exact paired inference and bootstrap CI: Because confirmatory outcomes are binary and the observed data are concentrated in concordant cells, we used an exact McNemar two-sided test on the discordant pair counts (n10 = 3, n01 = 0). The archived exact McNemar two-sided p = 0.25. The paired bootstrap percentile 95% CI for the paired-success-rate difference (computed with bootstrap_seed = 1729 and 10,000 resamples as recorded in analysis/analysis_log.jsonl) is [0.0000, 0.005838198498748957]. The CI lower bound reaching 0.0 reflects the high proportion of concordant successful pairs and the small number of discordant pairs; those data characteristics limit resolution for small absolute effects.

Missingness and failed calls: The preregistered missingness summary (analysis/missingness_summary.csv) reports complete_pairs = 1,199; failed_baseline_episodes = 1; failed_guarded_episodes = 1; both_missing_pairs = 1. The deterministic exclusion rules were not triggered for contamination; analysis/contamination_summary.csv is empty. The single paired exclusion implied by both_missing_pairs = 1 is the accounting source for the one planned pair drop from 1,200 to 1,199 complete pairs.

Repair invocation diagnostics: Stage_counts (execution manifest and deterministic_reconciliation.json) show repair = 3 while shared_initial_generation = 1,200. The repair count matches repair_provider traces present for exactly three episodes in the execution/provider_calls and execution/scoring artifacts. Scorer and validator traces (examples in execution/scoring/*.json) report validation_after.valid = true for the majority of episodes and repair_applied = false, indicating that initial candidates typically satisfied deterministic_netops_validator_v5 checks and therefore required no automated repair.

Representative artifact evidence: Representative task manifest entries and scoring artifacts illustrate the evidence pattern. For example, execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json show identical shared_initial_candidate content and validator traces with validation_after.valid = true and repair_applied = false; difficulty for that task is labeled "medium" in the task_manifest entry. Examples with higher difficulty labels ("hard", "very_hard") appear in representative task entries (task-000002 level "hard", task-000003 level "hard") and still produced accepted shared initial candidates in many cases. These artifact-level diagnostics demonstrate (a) heterogeneous declared task difficulty across the corpus, and (b) that under the chosen deterministic task generator and validator settings the initial LLM output frequently met intent constraints.

Interpretation of discordant pattern: The observed discordant pattern (n10 = 3, n01 = 0) means all discordant pairs favored the guarded pipeline; however, the absolute discordant count (3/1,199 $\approx$ 0.25%) is too small to yield statistical significance under the prespecified inferential thresholds. Under the exact McNemar framework the two-sided p = 0.25 (archived), reflecting the binomial probability of observing such an imbalanced discordant split given the small number of discordant pairs. The paired bootstrap CI's narrow upper bound ($\approx$0.00584) but lower bound at zero quantifies that any true absolute benefit, if present, is small in magnitude within this corpus and validator configuration.

Reproducibility anchors: All numbers above match archived artifacts: analysis/paired_contingency_table.csv (n11/n10/n01/n00), analysis/condition_summary.csv (per-condition counts and success rates), analysis/results.json (estimate, p-value, CI, bootstrap parameters), execution/execution_manifest.json (provider audit counts and timestamps), and representative scoring artifacts (execution/scoring/*.json) that record validator trace fields validation_before/validation_after, repair_applied flags, and normalized_configuration values. The run's bootstrap parameters (seed = 1729, resamples = 10,000) and analysis log are included in analysis/analysis_log.jsonl to enable exact reproduction of the CI reported above.

V. Discussion

Statistical and operational interpretation: The exact McNemar test (p = 0.25) and the paired bootstrap CI [0.0000, 0.0058381985] both indicate the observed guarded advantage is not distinguishable from zero under our confirmatory criterion. The empirical discordant counts (n10 = 3, n01 = 0) directly determine the paired estimate: paired_difference = 3/1,199 = 0.0025020851. Because the preregistered MDE was 0.05 (5 percentage points), the study—while sized for that target—was not powered to detect the much smaller absolute difference we observed. Concretely, observing a 5 pp absolute paired difference on the realized complete_pair_count (1,199) would require approximately 60 discordant pairs ($0.05 \times 1{,}199 \approx 59.95$), but we observed only three discordants; this arithmetic shows why the planned MDE and realized discordant count are inconsistent with detecting the tiny observed effect.

Artifact-grounded diagnostics explain why repairs were rare. The provider audit and stage counts record only three repair invocations (repair = 3) and model_calls_used = 1,203, corroborated by the validator traces where validation_after.valid = true and repair_applied = false for representative tasks (see execution/scoring/task-000001-baseline.json). Two interacting design choices reduced headroom for verifier benefit: (1) the synthetic task generator produced well-specified intents with canonical required_sequences (deterministic_netops_task_generator_v5), and (2) the deterministic validator (deterministic_netops_validator_v5) used parsing and constraint checks that accepted many reasonable initial candidates. These two factors are visible in representative task_manifest entries (required_sequence and difficulty fields) and in the scoring artifacts where parsed_command_count == required_command_count and violation_count == 0 for typical episodes. Together they explain the low frequency of repair-triggered iterations and the near-ceiling baseline success ($\approx$99.75%).

Reproducibility and forensic trace evidence: All numerical claims above are reproducible from archived artifacts. The confirmatory analysis used bootstrap_seed = 1729 with 10,000 resamples to compute the percentile CI (analysis/analysis_log.jsonl), and the paired contingency table is published (analysis/paired_contingency_table.csv). Execution accounting (execution/execution_manifest.json) reports started_at_utc and completed_at_utc, model_calls_used, completed pairs, and failed episodes. Artifact_count = 8,408 and representative scoring JSON entries include validator_trace objects showing validation_before/after, observed_touched_field_count, and repair_applied flags. Those deterministic traces permit exact reproduction of the numeric summaries without human adjudication.

Threats to validity made concrete by artifacts: Internal validity is high because task generation, seeds, and deterministic validators were frozen in the preregistration (preregistration SHA-256 = 77abe8b4...5712) and automation enforced exclusion rules; the missingness_summary.csv documents the two failed episodes and the single both_missing_pair. Construct validity depends on how well the binary episode success rule (validator acceptance + emulator availability tolerance + no rollback) maps to operator-valued correctness; the validator's acceptance thresholds and the emulator's availability tolerance are parameters embodied in the validator traces and transformation_manifest and therefore materially affect measured outcomes. External validity is limited: only one model family (requested_model = gpt-5-mini) was invoked (execution/model_configuration.json), tasks were synthetic/consented and vendor-agnostic, and emulation is Mininet-style rather than vendor-specific hardware; these constraints appear in task_manifest entries and in the emulator preflight logs. Conclusion validity is constrained by the small number of discordant pairs: standard paired tests rely on sufficient discordance to support inference, and here the discordant mass (3/1,199) is too small to yield narrow hypothesis tests for small absolute differences.

Implications for benchmark design and operational practice: The artifacts show that when initial generation already satisfies deterministic verifier rules for well-specified synthetic tasks, guarded architectures will seldom be exercised, and measured binary success gains will be tiny. To reveal verifier value in future studies, designers should (a) increase nominal task ambiguity or adversarial content so that initial candidates more frequently violate intent constraints, (b) vary validator strictness to control repair invocation rates, and (c) test multiple model families or fine-tuned variants to measure model–validator interactions. The archived run provides concrete knobs: change task_manifest difficulty fields, adjust validator violation thresholds, or expand maximum_repair_calls_per_task to study repair dynamics—each change is reproducible because seeds, manifests, and validator traces are preserved.

Operational diagnostics and sensitivity: The low repair count (3) correlated with specific task patterns (deterministic task difficulty levels labelled "medium" or "hard" in representative manifests) but was insufficient to permit reliable subgroup inference. Exploratory artifacts (analysis/condition_summary.csv and analysis/missingness_summary.csv) can be used to stratify by difficulty and re-estimate discordant rates; however, such exploratory subgroup tests remain underpowered without increasing task diversity or sample size. The run's provider audit also shows a single failed hosted_model_call_event_count = 1; recovery and retry rules are logged and the stopping_rule was enforced without manual intervention.

Concluding operational guidance: The null confirmatory result—guarded advantage not statistically significant—should be interpreted as conditional on the task corpus, validator configuration, and requested model. The run artifacts consistently point to limited headroom (near-ceiling baseline) and low repair invocation as the proximate causes. For practitioners, the key takeaway is not that verifiers are universally unnecessary, but rather that their measured value depends strongly on task ambiguity and validator policy. We provide the full artifact set (task manifests, validator traces, provider audits, analysis logs, and seeds) to enable sensitivity experiments that vary these parameters and thereby expose conditions under which verifier scaffolding yields operationally meaningful improvements.

VI. Limitations

- Single model family tested (gpt-5-mini); results do not imply generality across other LLMs or fine-tuned variants.
- Task corpus skew: synthetic/consented tasks yielded near-ceiling baseline success ($\approx$99.75%), limiting headroom and statistical power to detect practical improvements by guarded pipelines.
- Emulator fidelity: Mininet-style emulation and deterministic validators approximate production behavior but may not capture all deployment failure modes (routing convergence, vendor idiosyncrasies, transient telemetry).
- Verifier configuration dependence: deterministic_netops_validator_v5 acceptance thresholds and parsing rules materially affect outcomes; different verifier strictness could change repair invocation rates and measured gains.
- Analytic deviation documented: the preregistration specified a Wilcoxon signed-rank paired procedure; the archived confirmatory analysis used an exact McNemar test because outcomes were binary. This deviation is recorded in the archived analysis artifacts and in the Disclosure Statement above.
- Operational external validity: absence of heterogeneous vendor renderers, real production templates, and live rollouts constrain direct production generalization.

VII. Conclusion

We preregistered and executed NetOpsTraceBench, a paired, emulator-grounded study comparing one-shot and verifier-assisted NL$\rightarrow$configuration pipelines on 1,199 completed pairs. Under our synthetic task corpus, deterministic validator settings, and gpt-5-mini as requested, baseline one-shot generation already achieved near-ceiling success ($\approx$99.75%). The guarded pipeline increased absolute success by $\approx$0.25 percentage points (paired difference = 0.0025020851) with exact McNemar two-sided p = 0.25 and a 95% bootstrap CI that includes zero. Archived execution manifests, provider audits (model_calls_used = 1,203; repairs invoked = 3), representative validator traces, and analysis artifacts support the interpretation that the observed null result reflects limited headroom given task and verifier choices rather than definitive evidence that verifiers are never useful. Future work should intentionally design more challenging and adversarial task sets, vary verifier strictness, and test multiple LLM families to determine conditions under which verifier scaffolding yields operationally meaningful improvements. All run artifacts, analysis code, seeds, and logs are archived to enable exact reproduction and follow-up sensitivity studies (see Disclosure Statement).

Disclosure Statement

Mandatory disclosure (preregistered run rX4f7-1). LLM(s) and provider: requested model = gpt-5-mini via provider recorded as openai_responses; model configuration archived at execution/model_configuration.json (requested_model=gpt-5-mini, max_output_tokens=1500). Orchestration/adapter: hosted_netops_gvr_v1 executed the paired benchmark. Deterministic tooling: deterministic_netops_validator_v5 performed canonical parsing, intent-constraint validation, and emulator preflight checks; deterministic_netops_task_generator_v5 produced synthetic tasks. Exact initial master prompt: archived at provenance/master_prompt.txt (immutable reference SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5a cdf1582bb39b15ba). Topic constraint and autonomy boundary: the human Research Director selected the topic family and approved the master prompt pre-lock; after the autonomy lock the AI autonomously formulated the research question, conducted literature review, generated the preregistration, executed the experiment, performed analysis, and wrote and revised the manuscript. Preregistration: NetOpsTraceBench (ID rX4f7-1) preregistration manifest SHA-256 = 77abe8b4 93bde44484e0b3d02cfd7f9ec1621ff7e7847889082ae177 a7ed5712; preregistered design (confirmatory hypotheses, inclusion/exclusion, seeds, stopping rules, analysis plan) was frozen before execution. Execution summary: started 2026-08-30T01:21:55.415955+00:00, completed 2026-08-30T03:15:12.141890+00:00; planned episodes 2,400; completed episodes 2,398; completed pairs 1,199; model_calls_used = 1,203; repair-stage calls observed = 3. Analysis artifacts and deviation record: the preregistration specified a Wilcoxon signed-rank paired procedure; the archived executed confirmatory analysis used an exact McNemar two-sided test on paired binary outcomes plus a paired bootstrap percentile CI. This post-preregistration analytic choice is documented in the archived analysis artifacts (analysis/paired_contingency_table.csv, analysis/results.json, analysis/analysis_log.jsonl, analysis/paired_difference_figure.svg) produced as part of the run that completed at 2026-08-30T03:15:12.141890+00:00. Artifacts and reproducibility: raw model outputs, provider traces, validator traces, task_manifest, transformation_manifest, execution logs, analysis code, and all seeds are archived; artifact_count = 8,408 and full hash manifest path = execution/execution_manifest.json. Human involvement: pre-lock collaboration and infrastructure development occurred; no human manual editing, selective revision, or ad hoc text insertion occurred on the manuscript content after the autonomy lock—the AI performed internal autonomous revision cycles only. Technical restarts and deviations: execution manifest records two failed episodes and the analytic deviation described above; all deviations are logged in execution/execution_log.jsonl. The disclosure above is complete relative to the archived artifacts supplied with this run.

References

[1] http://hdl.handle.net/20.500.12708/229494
[2] https://arxiv.org/abs/2608.13389
[3] http://arxiv.org/abs/2403.09369
[4] https://doi.org/10.2139/ssrn.6947162
[5] https://doi.org/10.2139/ssrn.7222278
[6] https://doi.org/10.20935/acadai8260
[7] https://doi.org/10.3390/info17030297
[8] https://doi.org/10.3390/app16010085